AI-Powered Interview Assistant — Project Code

Formatted PDF with Code Blocks

File: README.md

```
# AI-Powered Interview Assistant — Project Code

This PDF contains the code for the React project implementing the Swipe Internship Assignment: AI-Powered Interview Assistant.

Structure (files included in this PDF):
- package.json
- public/index.html
- src/index.jsx
- src/App.jsx
- src/store.js
- src/slices/candidatesSlice.js
- src/components/ResumeUploader.jsx
- src/components/Chat.jsx
- src/components/Dashboard.jsx
- src/utils/pdfParser.js
- src/services/aiService.js
- src/styles.css

Note: The AI integration is mocked (aiService) — replace with an API (OpenAI, etc.) as required.
```

File: package.json

```
{
  "name": "swipe-interview-assistant",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "vite",
    "build": "vite build",
    "serve": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-redux": "^8.1.0",
    "@reduxjs/toolkit": "^1.9.0",
    "redux-persist": "^6.0.0",
    "pdfjs-dist": "^3.6.172",
    "file-saver": "^2.0.5"
  },
  "devDependencies": {
    "vite": "^5.0.0",
    "@vitejs/plugin-react": "^4.0.0"
  }
}
```

File: public/index.html

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>AI Interview Assistant</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/index.jsx"></script>
  </body>
</html>
```

File: src/index.jsx

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { PersistGate } from 'redux-persist/integration/react';
import App from './App';
import { store, persistor } from './store';
import './styles.css';

createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <PersistGate loading={null} persistor={persistor}>
      <App />
    </PersistGate>
  </Provider>
);
```

File: src/App.jsx

```
import React, { useState } from 'react';
import ResumeUploader from './components/ResumeUploader';
import Chat from './components/Chat';
import Dashboard from './components/Dashboard';
import { useSelector } from 'react-redux';

export default function App(){
  const [tab, setTab] = useState('interviewee');
  const candidates = useSelector(s => s.candidates.list);

  return (
    <div className="app">
      <header>
        <h1>AI Interview Assistant</h1>
        <nav>
          <button onClick={() => setTab('interviewee')} className={tab==='interviewee'?'active':''}>Interv
iewee</button>
          <button onClick={() => setTab('interviewer')} className={tab==='interviewer'?'active':''}>Interv
iewer</button>
        </nav>
      </header>
      <main>
        {tab === 'interviewee' ? (
          <div className="panel">
            <ResumeUploader />
            <Chat />
          </div>
        ) : (
          <Dashboard candidates={candidates} />
        )}
      </main>
    </div>
  );
}
```

File: src/store.js

```
import { configureStore } from '@reduxjs/toolkit';
import { combineReducers } from 'redux';
import candidatesReducer from './slices/candidatesSlice';
import {
  persistStore,
  persistReducer,
  FLUSH,
  REHYDRATE,
```

```
  PAUSE,
  PERSIST,
  PURGE,
  REGISTER,
} from 'redux-persist';
import storage from 'redux-persist/lib/storage';

const rootReducer = combineReducers({
  candidates: candidatesReducer,
});

const persistConfig = {
  key: 'root',
  storage,
};

const persisted = persistReducer(persistConfig, rootReducer);

export const store = configureStore({
  reducer: persisted,
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware({
      serializableCheck: {
        ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER],
      },
    }),
});

export const persistor = persistStore(store);
```

File: src/slices/candidatesSlice.js

```
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  list: []
};

function scoreFromAnswers(answers){
  // Simple scoring: easy 0-10, medium 0-20, hard 0-30 (weights)
  let score = 0;
  for(const a of answers){
    const { difficulty, aiScore=5 } = a;
    if(difficulty==='easy') score += (aiScore / 10) * 10;
    if(difficulty==='medium') score += (aiScore / 20) * 20;
    if(difficulty==='hard') score += (aiScore / 30) * 30;
  }
  return Math.round(score);
}

const slice = createSlice({
  name: 'candidates',
  initialState,
  reducers:{
    upsertCandidate(state, action){
      const c = action.payload;
      const idx = state.list.findIndex(x => x.id === c.id);
      if(idx === -1) state.list.push(c);
      else state.list[idx] = c;
    },
    addAnswer(state, action){
      const {id, answer} = action.payload;
      const cand = state.list.find(x => x.id === id);
      if(!cand) return;
      cand.answers = cand.answers || [];
```

```
      cand.answers.push(answer);
      // update score and summary after last question
      if(cand.answers.length === 6){
        cand.score = scoreFromAnswers(cand.answers);
        cand.summary = 'Auto-generated summary (replace with AI). Final score: ' + cand.score;
      }
    },
    createCandidate(state, action){
      const c = action.payload;
      state.list.push(c);
    }
  }
});

export const { upsertCandidate, addAnswer, createCandidate } = slice.actions;
export default slice.reducer;
```

File: src/components/ResumeUploader.jsx

```
import React, { useRef, useState } from 'react';
import { useDispatch } from 'react-redux';
import pdfParser from '../utils/pdfParser';
import { upsertCandidate, createCandidate } from '../slices/candidatesSlice';
import { v4 as uuidv4 } from 'uuid';

export default function ResumeUploader(){
  const fileRef = useRef();
  const [msg, setMsg] = useState('');
  const dispatch = useDispatch();

  async function handleFile(e){
    const f = e.target.files[0];
    if(!f) return;
    const name = f.name.toLowerCase();
    if(!name.endsWith('.pdf') && !name.endsWith('.docx')){
      setMsg('Please upload PDF or DOCX');
      return;
    }
    setMsg('Parsing...');
    try{
      const parsed = await pdfParser(f);
      const id = uuidv4();
      const cand = {
        id,
        name: parsed.name || '',
        email: parsed.email || '',
        phone: parsed.phone || '',
        resumeName: f.name,
        answers: [],
        createdAt: Date.now()
      };
      dispatch(createCandidate(cand));
      setMsg('Candidate created. Fill missing fields in chat if any.');
    }catch(err){
      console.error(err);
      setMsg('Failed to parse resume.');
    }
  }

  return (
    <div className="uploader">
      <h2>Upload Resume (PDF/DOCX)</h2>
      <input ref={fileRef} type="file" accept=".pdf,.docx" onChange={handleFile} />
      <div>{msg}</div>
    </div>
```

```
  );
}
```

File: src/components/Chat.jsx

```
import React, { useEffect, useState, useRef } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { addAnswer, upsertCandidate } from '../slices/candidatesSlice';
import aiService from '../services/aiService';

const QUESTIONS_BY_DIFFICULTY = {
  easy: [
    'What is the difference between let and var in JavaScript?',
    'Explain CSS box model.'
  ],
  medium: [
    'Explain component lifecycle in React and hooks alternatives.',
    'How does event loop work in Node.js?'
  ],
  hard: [
    'Design a scalable authentication system for microservices.',
    'Explain strategies for database sharding and partitioning.'
  ]
};

export default function Chat(){
  const candidates = useSelector(s => s.candidates.list);
  const [currentId, setCurrentId] = useState(candidates.length? candidates[0].id : null);
  const [messages, setMessages] = useState([]);
  const [questionIndex, setQuestionIndex] = useState(0);
  const [phase, setPhase] = useState('idle'); // idle | interviewing | finished
  const [timer, setTimer] = useState(0);
  const [remaining, setRemaining] = useState(0);
  const answerRef = useRef();
  const dispatch = useDispatch();

  useEffect(()=>{
    if(candidates.length && !currentId) setCurrentId(candidates[0].id);
  }, [candidates]);

  useEffect(()=>{
    let t;
    if(phase === 'interviewing' && remaining > 0){
      t = setInterval(()=> setRemaining(r => r-1), 1000);
    }else if(phase === 'interviewing' && remaining === 0){
      // auto submit empty answer
      submitAnswer('');
    }
    return ()=> clearInterval(t);
  }, [phase, remaining]);

  function getDifficultyForIndex(i){
    if(i < 2) return 'easy';
    if(i < 4) return 'medium';
    return 'hard';
  }

  function startInterview(){
    if(!currentId) return alert('No candidate selected. Upload resume first.');
    setMessages([]);
    setQuestionIndex(0);
    askNextQuestion(0);
    setPhase('interviewing');
  }
```

```
  function askNextQuestion(i){
    const diff = getDifficultyForIndex(i);
    const q = QUESTIONS_BY_DIFFICULTY[diff][i % QUESTIONS_BY_DIFFICULTY[diff].length];
    setMessages(m => [...m, { from: 'ai', text: `(${diff.toUpperCase()}) Q${i+1}: ${q}`, difficulty: diff
}]);
    const t = diff === 'easy' ? 20 : diff === 'medium' ? 60 : 120;
    setTimer(t);
    setRemaining(t);
  }

  async function submitAnswer(text){
    const cand = candidates.find(c => c.id === currentId);
    const qmsg = messages[messages.length-1];
    setMessages(m => [...m, { from: 'user', text }]);
    // call AI to score
    const aiScore = await aiService.scoreAnswer(qmsg.text, text);
    const answerObj = {
      question: qmsg.text,
      answer: text,
      difficulty: qmsg.difficulty,
      aiScore
    };
    dispatch(addAnswer({ id: currentId, answer: answerObj }));
    if(questionIndex+1 >= 6){
      setPhase('finished');
      setMessages(m => [...m, { from:'ai', text: 'Interview finished. Calculating summary...' }]);
      // fake summary
      const score = (cand && cand.score) || 'Pending';
      setMessages(m => [...m, { from:'ai', text: `Final score: ${score}` }]);
      return;
    }else{
      const next = questionIndex + 1;
      setQuestionIndex(next);
      askNextQuestion(next);
    }
  }

  return (
    <div className="chat">
      <div className="chat-controls">
        <select value={currentId || ''} onChange={e => setCurrentId(e.target.value)}>
          <option value="">Select candidate</option>
          {candidates.map(c => <option key={c.id} value={c.id}>{c.name || c.email || c.resumeName}</option
>)}
        </select>
        <button onClick={startInterview}>Start Interview</button>
      </div>

      <div className="messages">
        {messages.map((m,i) => <div key={i} className={"msg "+m.from}><b>{m.from}</b>: {m.text}</div>)}
      </div>

      {phase === 'interviewing' && (
        <div className="answer-box">
          <div>Time remaining: {remaining}s</div>
          <textarea ref={answerRef} placeholder="Type your answer..."></textarea>
          <button onClick={()=> submitAnswer(answerRef.current.value)}>Submit</button>
        </div>
      )}
    </div>
  );
}
```

File: src/components/Dashboard.jsx

```
import React, { useState } from 'react';
import { useSelector } from 'react-redux';

export default function Dashboard(){
  const candidates = useSelector(s => s.candidates.list || []);
  const [query, setQuery] = useState('');
  const [sortBy, setSortBy] = useState('score');

  const filtered = candidates.filter(c => {
    if(!query) return true;
    return (c.name || '').toLowerCase().includes(query.toLowerCase()) ||
           (c.email || '').toLowerCase().includes(query.toLowerCase());
  }).sort((a,b) => (b.score || 0) - (a.score || 0));

  return (
    <div className="dashboard">
      <h2>Interviewer Dashboard</h2>
      <div className="controls">
        <input placeholder="Search by name/email" value={query} onChange={e=>setQuery(e.target.value)} />
        <select value={sortBy} onChange={e=>setSortBy(e.target.value)}>
          <option value="score">Sort by score</option>
          <option value="date">Sort by date</option>
        </select>
      </div>

      <table className="candidates-table">
        <thead><tr><th>Name</th><th>Email</th><th>Score</th><th>Summary</th></tr></thead>
        <tbody>
          {filtered.map(c => (
            <tr key={c.id}>
              <td>{c.name}</td>
              <td>{c.email}</td>
              <td>{c.score || '—'}</td>
              <td>{c.summary || '—'}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

File: src/utils/pdfParser.js

```
import pdfjsLib from 'pdfjs-dist';
import pdfjsWorker from 'pdfjs-dist/build/pdf.worker.entry';

pdfjsLib.GlobalWorkerOptions.workerSrc = pdfjsWorker;

async function parsePdfFile(file){
  if(!file) throw new Error('No file');
  const arrayBuffer = await file.arrayBuffer();
  const pdf = await pdfjsLib.getDocument({data: arrayBuffer}).promise;
  let text = '';
  for(let i=1;i<=pdf.numPages;i++){
    const page = await pdf.getPage(i);
    const content = await page.getTextContent();
    const strs = content.items.map(it => it.str).join(' ');
    text += ' ' + strs;
  }
  // simple extraction with regex
  const nameMatch = text.match(/([A-Z][a-z]+\s[A-Z][a-z]+)/);
  const emailMatch = text.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/);
```

```
  const phoneMatch = text.match(/(\+?\d[\d\s-]{7,}\d)/);
  return {
    name: nameMatch ? nameMatch[0] : '',
    email: emailMatch ? emailMatch[0] : '',
    phone: phoneMatch ? phoneMatch[0].replace(/\s+/g,'') : ''
  };
}

export default parsePdfFile;
```

File: src/services/aiService.js

```
// Mocked AI service. Replace with real API calls to OpenAI or other LLM provider.
const aiService = {
  async scoreAnswer(question, answer){
    // Basic heuristics: length + keywords
    if(!answer || answer.trim().length === 0) return 5;
    const len = answer.trim().length;
    let score = Math.min(30, Math.max(5, Math.floor(len / 10)));
    return score; // numeric score reflecting quality (higher better)
  },
  async generateSummary(candidate){
    return 'Candidate summary generated by AI mock. Replace with real API call.';
  }
};

export default aiService;
```

File: src/styles.css

```
body{ font-family: Arial, Helvetica, sans-serif; margin:0; padding:0; background:#f5f7fa; color:#0a0a0a;}
.app header{ display:flex; align-items:center; justify-content:space-between; padding:16px; background:#ff
f; border-bottom:1px solid #eee;}
.app header h1{ margin:0; font-size:18px;}
nav button{ margin-left:8px; padding:8px 12px; border:1px solid #ddd; background:transparent; cursor:point
er;}
nav button.active{ background:#0366d6; color:#fff; border-color:#0366d6;}
.main{ padding:16px;}
.uploader{ padding:12px; background:#fff; border:1px solid #eee; margin-bottom:12px;}
.chat{ padding:12px; background:#fff; border:1px solid #eee;}
.messages{ max-height:300px; overflow:auto; padding:8px; border:1px dashed #eee; margin-bottom:8px;}
.msg.ai{ color:#0366d6;}
.msg.user{ color:#333;}
.answer-box textarea{ width:100%; height:100px; }
```